

Aim: TypeScript Student Registration System

1. Classes and Objects

A class in TypeScript is a blueprint for creating objects. It encapsulates data (properties) and behavior (methods). In this project, we use classes to represent a "Person" and a "Student".

2. Inheritance

Inheritance allows a class (subclass) to acquire the properties and methods of another class (superclass). Here, the Student class extends the Person class, demonstrating the "is-a" relationship (a Student is a Person).

3. Access Modifiers

Access modifiers control the visibility of class members:

- Public: Accessible from anywhere (e.g., Student name/age).
- Private: Accessible only within the class it is defined (e.g., Roll Number for security).
- Protected: Accessible within the class and its subclasses (e.g., internal Person identifiers).

Code (index.html):

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta
      name="viewport"
      content="width=device-width,
initial-scale=1.0, user-scalable=yes"
    />
    <title>TypeScript Student
Registration System</title>
  </head>
  <body>
    <div class="app-container">
      <div class="header">
        <h1>Student Registration
System</h1>
      </div>
      <div class="dashboard">
        <div class="registration-
panel">
          <h2 class="panel-
title">Register new student</h2>
          <form
            id="studentForm">
            <div class="form-
group">
              <label> Full
name</label>
              <input
                type="tex
t"
                id="stude
ntName"
                placehold
er="e.g., Priya Sharma"
                autocompl
ete="off"
                required
              />
            </div>
            <div class="form-
group">
              <label>
Student ID</label>
```

```

        <input
            type="text"
            id="studentId"
            placeholder="e.g., S12345"
            autocomplete="off"
            required
        />
    </div>
    <div class="form-group">
        <label> Email address</label>
        <input
            type="email"
            id="studentEmail"
            placeholder="student@example.com"
            autocomplete="off"
            required
        />
    </div>
    <div class="form-group">
        <label> Course</label>
        <select
            id="studentCourse">
                <option
                    value="Computer Science">
                        Computer Science
                </option>
                <option
                    value="Business Administration">
                        Business Administration
            </select>
    </div>
    <div class="student-list-panel">
        <h2 class="panel-title">
            Registered Students
            <span
                id="studentCountBadge" class="stats-badge">
                    >0</span>
        </h2>
        <div
            id="studentsContainer">
            <div
                class="empty-message">
                    No students yet. Register your first student!
            </div>
        </div>
    </div>
</div>
<script src="index.ts"></script>
</body>
</html>

```

Code (index.ts)

```

class StudentManager {
    #students = [];

    #container;

    #countBadge;

    constructor(containerId, badgeId) {
        const container =
            document.getElementById(containerId);

        const badge =
            document.getElementById(badgeId);

        if (!container)
            throw new Error(
                `Container element with
                id "${containerId}" not found`,
            );
        if (!badge)

```

```

        throw new Error(`Badge
element with id "${badgeId}" not found`);

        this.#container = container;
        this.#countBadge = badge;
    }

    getAllStudents() {
        return [...this.#students];
    }

    addStudent(studentData) {
        const duplicate =
this.#students.some(
            (s) =>
                s.studentId.toLowerCase()
===
                studentData.studentId.toL
owerCase(),
            );
        if (duplicate) {
            return {
                success: false,
                message: `Student with ID
"${studentData.studentId}" already
exists.` ,
            };
        }

        const newId =
`student_${Date.now()}_${Math.random().to
String(36).substring(2, 8)}`;

        const newStudent = {
            id: newId,
            name:
studentData.name.trim(),
            studentId:
studentData.studentId.trim(),
            email:
studentData.email.trim(),
            course: studentData.course,
        };

        this.#students.push(newStudent);
        this.#renderStudentList();
        return {
            success: true,
            message: "Student added
successfully!",
        };
    }

    removeStudentById(internalId) {
        const initialLength =
this.#students.length;

```

```

        this.#students =
this.#students.filter(
            (student) => student.id !==
internalId,
        );
        const removed = initialLength !==
this.#students.length;
        if (removed) {
            this.#renderStudentList();
        }
        return removed;
    }

    #renderStudentList() {
        const count =
this.#students.length;
        this.#countBadge.textContent =
String(count);

        if (count === 0) {
            this.#container.innerHTML =
`<div class="empty-message"> No
registered students yet. Use the form to
add.</div>`;
            return;
        }

        const cardsHtml = this.#students
            .map((student) => {
                const safeName =
this.#escapeHtml(student.name);
                const safeStudentId =
this.#escapeHtml(student.studentId);
                const safeEmail =
this.#escapeHtml(student.email);
                const safeCourse =
this.#escapeHtml(student.course);

                return `
                    <div class="student-
card" data-student-
id="${this.#escapeHtml(student.id)}">
                        <div
class="student-info">
                            <div
class="student-name">${safeName}</div>
                            <div
class="student-details">
                                <span>
${safeStudentId}</span>
                                <span>
${safeEmail}</span>
                                <span>
${safeCourse}</span>
                            </div>
                        </div>

```

```

        <button
class="remove-btn" data-remove-
id="${this.#escapeHtml(student.id)}"
aria-label="Remove student">
Remove</button>
    </div>
    `;
    })
    .join("");

    this.#container.innerHTML =
cardsHtml;

    const removeButtons =
this.#container.querySelectorAll(".remove-
-btn");
    removeButtons.forEach((btn) => {
        btn.addEventListener("click",
(event) => {
            event.stopPropagation();
            const internalId =
btn.getAttribute("data-remove-id");
            if (internalId) {
                const removed =
this.removeStudentById(internalId);
                if (removed) {
                    this.#showToast(
                        `Student
removed successfully`,
                        "#10b981",
                    );
                } else {
                    this.#showToast(
                        `Could not remove student`, "#ef4444");
                }
            }
        });
    });

    #escapeHtml(str) {
        if (!str) return "";
        return str
            .replace(/[<>]/g, function
(m) {
                if (m === "&") return
"&amp;";
                if (m === "<") return
"&lt;";
                if (m === ">") return
"&gt;";
            })
            .replace(/[\uD800-
\uDBFF][\uDC00-\uDFFF]/g, function (c) {
                return c;
            });
    }

```

```

    });
}

    #showToast(message, bgColor =
"#3b82f6") {
        let toast =
document.getElementById("dynamicToast");
        if (!toast) {
            toast =
document.createElement("div");
            toast.id = "dynamicToast";
            toast.style.position =
"fixed";
            toast.style.bottom = "20px";
            toast.style.left = "50%";
            toast.style.transform =
"translateX(-50%)";
            toast.style.backgroundColor =
bgColor;
            toast.style.color = "white";
            toast.style.padding = "12px
24px";
            toast.style.borderRadius =
"40px";
            toast.style.fontWeight =
"500";
            toast.style.fontSize =
"0.9rem";
            toast.style.zIndex = "9999";
            toast.style.boxShadow = "0
10px 20px rgba(0,0,0,0.2)";
            toast.style.backdropFilter =
"blur(4px)";
            toast.style.transition =
"opacity 0.2s";
            document.body.appendChild(toa
st);
        }
        toast.style.backgroundColor =
bgColor;
        toast.textContent = message;
        toast.style.opacity = "1";
        setTimeout(() => {
            toast.style.opacity = "0";
            setTimeout(() => {
                if (toast.parentNode)
toast.remove();
            }, 300);
        }, 2000);
    }

    class RegistrationFormController {
        constructor(manager, formId) {
            this.#manager = manager;
        }
    }

```

```

        const formElement =
document.getElementById(formId);
        if (!(formElement instanceof
HTMLFormElement)) {
            throw new Error(
                `Form with id "${formId}"
not found or not a form element.`
            );
        }
        this.#form = formElement;

        const nameInput =
document.getElementById("studentName");
        const studentIdInput =
document.getElementById("studentId");
        const emailInput =
document.getElementById("studentEmail");
        const courseSelect =
document.getElementById("studentCourse");

        if (!nameInput || !studentIdInput
|| !emailInput || !courseSelect) {
            throw new Error("Required
form fields missing from DOM");
        }

        this.#nameInput = nameInput;
        this.#studentIdInput =
studentIdInput;
        this.#emailInput = emailInput;
        this.#courseSelect =
courseSelect;

        this.#attachEvent();
    }

    #attachEvent() {
        this.#form.addEventListener("subm
it", (event) => {
            event.preventDefault();
            this.#handleAddStudent();
        });
    }

    #handleAddStudent() {
        let name =
this.#nameInput.value.trim();
        let studentId =
this.#studentIdInput.value.trim();
        let email =
this.#emailInput.value.trim();
        let course =
this.#courseSelect.value;

        if (name === "") {

```

```

            this.#showFieldError("Please
enter student name", this.#nameInput);
            return;
        }
        if (studentId === "") {
            this.#showFieldError(
                "Student ID is required",
                this.#studentIdInput,
            );
            return;
        }
        if (email === "") {
            this.#showFieldError("Email
address is required", this.#emailInput);
            return;
        }

        const emailPattern =
/^[\^s@]+@([\^s@.,]+\.[\^s@.,]{2,})$/;
        if (!emailPattern.test(email)) {
            this.#showFieldError(
                "Please enter a valid
email (e.g., name@domain.com)",
                this.#emailInput,
            );
            return;
        }

        const idPattern = /^[a-zA-Z0-9_-
]+$/;
        if (!idPattern.test(studentId)) {
            this.#showFieldError(
                "Student ID can only
contain letters, numbers, underscores or
hyphens.",
                this.#studentIdInput,
            );
            return;
        }

        const result =
this.#manager.addStudent({
            name,
            studentId,
            email,
            course,
        });

        if (result.success) {
            this.#nameInput.value = "";
            this.#studentIdInput.value =
            "";
            this.#emailInput.value = "";
            this.#courseSelect.selectedIn
dex = 0;

```

```

        this.#clearFieldHighlights();

        this.#showFloatingMessage(`${
result.message}`, "#3b82f6");
    } else {
        this.#showFloatingMessage(
`${result.message} |
Duplicate or invalid data}`,
        "#ef4444",
        );

        if
(result.message?.includes("already
exists")) {
            this.#highlightField(this
.#studentIdInput);
        }
    }

    #showFieldError(message,
inputElement) {
        this.#showFloatingMessage(message
, "#f97316");
        this.#highlightField(inputElement
);
    }

    #highlightField(input) {
        input.style.border = "2px solid
#ef4444";
        input.style.backgroundColor =
"#fff5f5";
        setTimeout(() => {
            input.style.border = "";
            input.style.backgroundColor =
"";
        }, 2000);
    }

    #clearFieldHighlights() {
        [this.#nameInput,
this.#studentIdInput,
this.#emailInput].forEach(
        (inp) => {
            inp.style.border = "";
            inp.style.backgroundColor
= "";
        },
        );
    }

    #showFloatingMessage(msg, bgColor) {
        let toast =
document.getElementById("dynamicToast");
        if (!toast) {

```

```

            toast =
document.createElement("div");
            toast.id = "dynamicToast";
            toast.style.position =
"fixed";
            toast.style.bottom = "20px";
            toast.style.left = "50%";
            toast.style.transform =
"translateX(-50%)";
            toast.style.padding = "12px
24px";
            toast.style.borderRadius =
"40px";
            toast.style.fontWeight =
"500";
            toast.style.fontSize =
"0.9rem";
            toast.style.zIndex = "9999";
            toast.style.boxShadow = "0
8px 20px rgba(0,0,0,0.2)";
            toast.style.transition =
"opacity 0.2s";
            toast.style.color = "white";
            document.body.appendChild(toa
st);
        }
        toast.style.backgroundColor =
bgColor;
        toast.textContent = msg;
        toast.style.opacity = "1";
        setTimeout(() => {
            toast.style.opacity = "0";
            setTimeout(() => {
                if (toast.parentNode &&
toast.id === "dynamicToast") {
                    if
(toast.style.opacity === "0")
toast.remove();
                }
            }, 300);
        }, 2200);
    }

    document.addEventListener("DOMContentLoad
ed", () => {
        try {
            const studentManager = new
StudentManager(
                "studentsContainer",
                "studentCountBadge",
            );

            const formController = new
RegistrationFormController(
                studentManager,

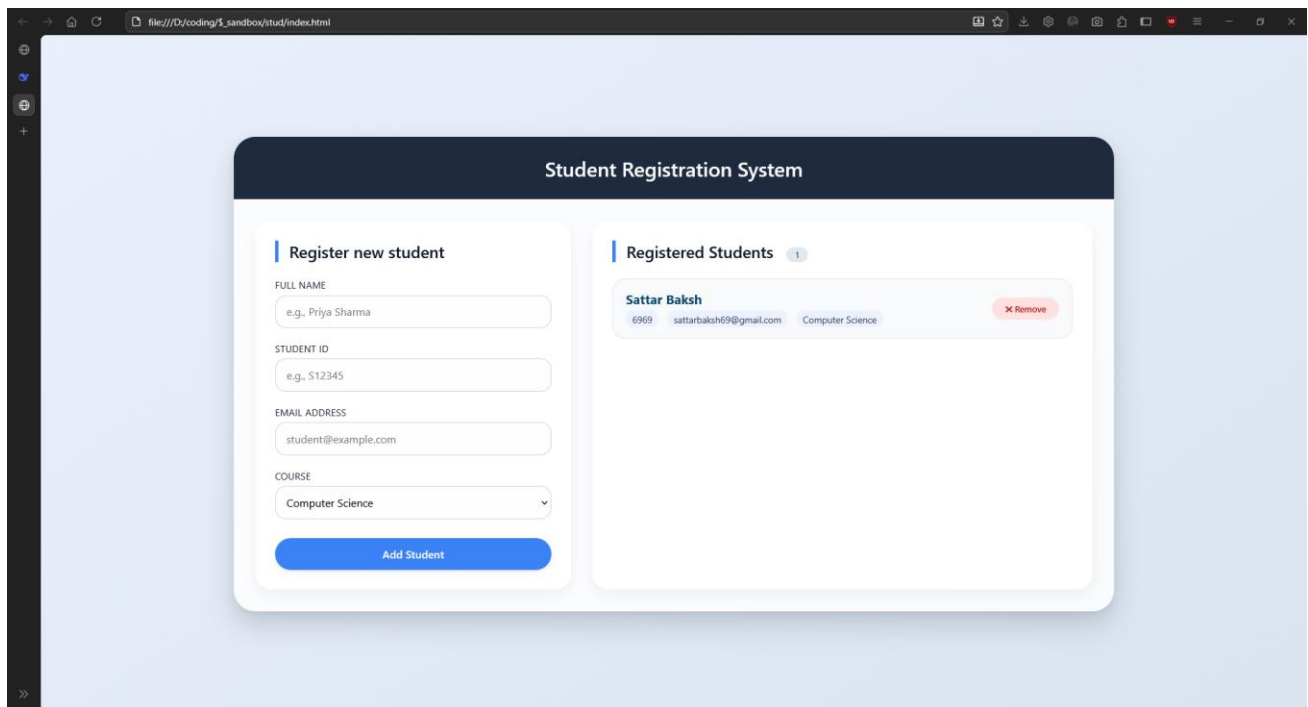
```

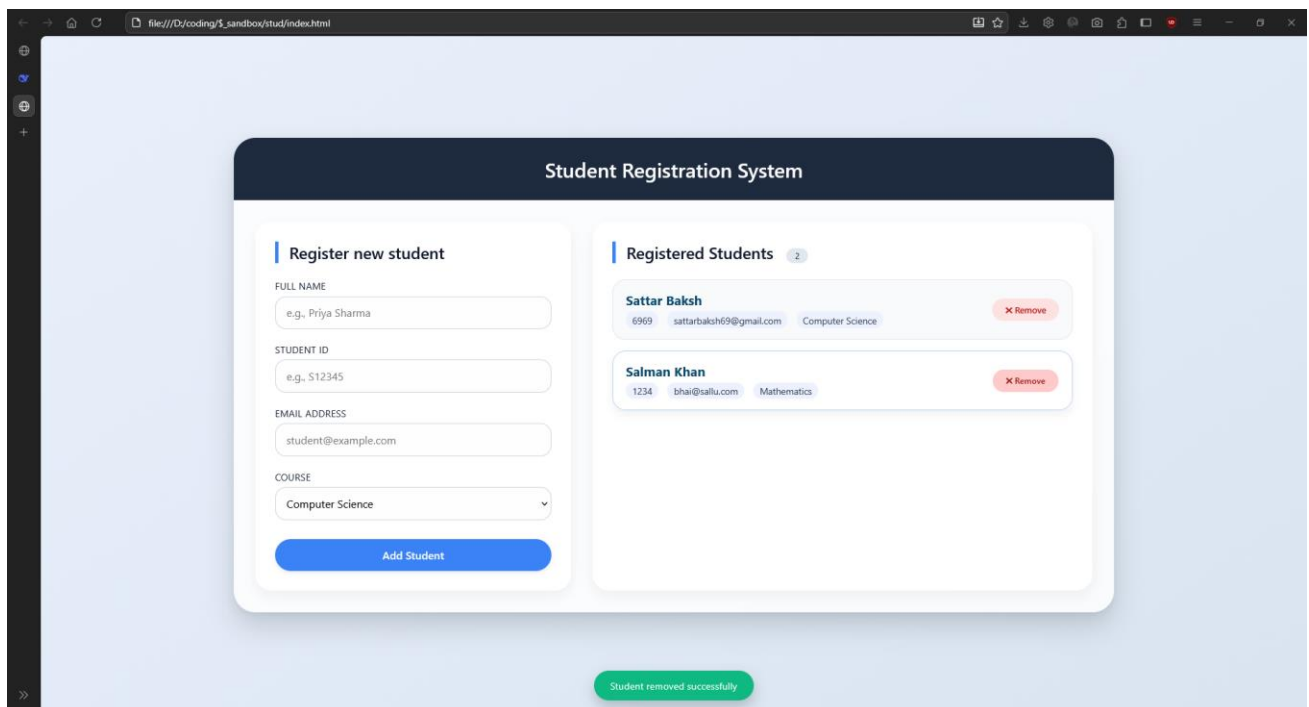
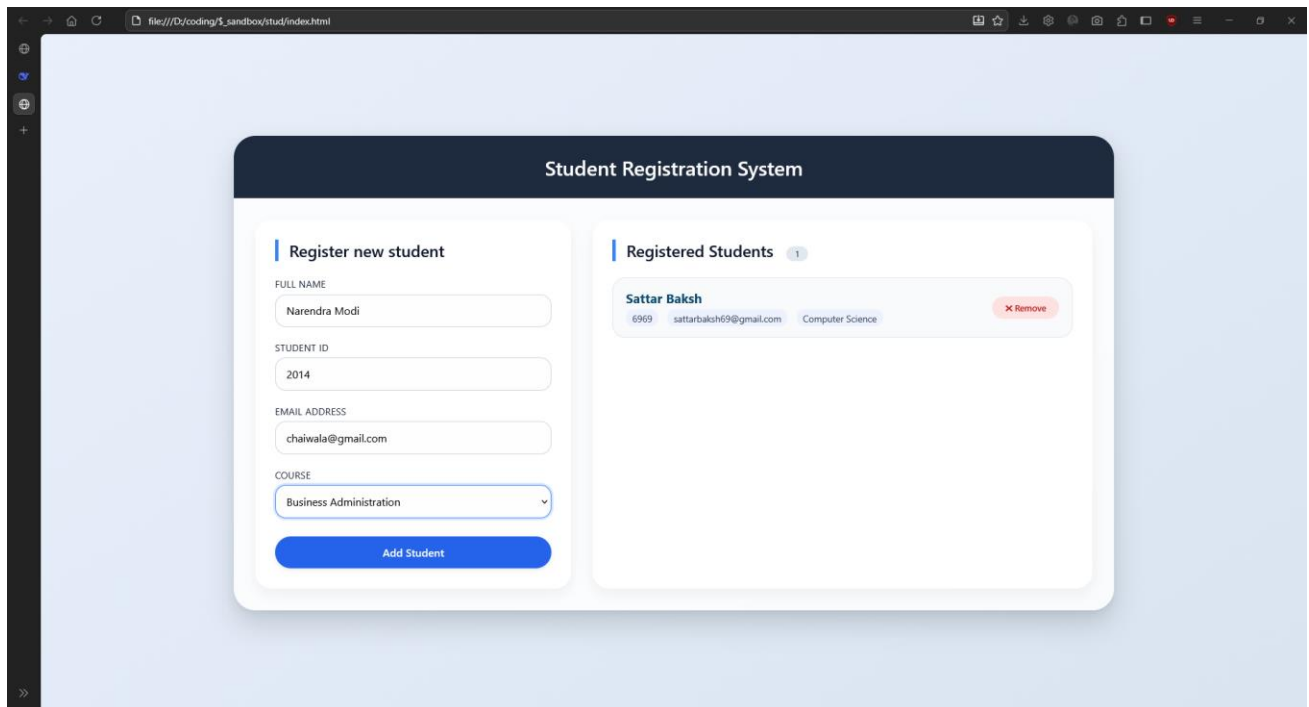
```
        "studentForm",
    );

    } catch (error) {
        console.error("Initialization
error:", error);
        const container =
document.getElementById("studentsContaine
r");

        if (container) {
            container.innerHTML = `<div
class="empty-message"
style="color:red;">System failed to load:
${error.message}</div>`;
        }
    }
});
```

Output:





Conclusion:

The application successfully demonstrates the use of TypeScript's strictly typed nature and OOP features. By using inheritance, we reduced code redundancy, and access modifiers ensured better data encapsulation, making the registration system robust and easy to scale.